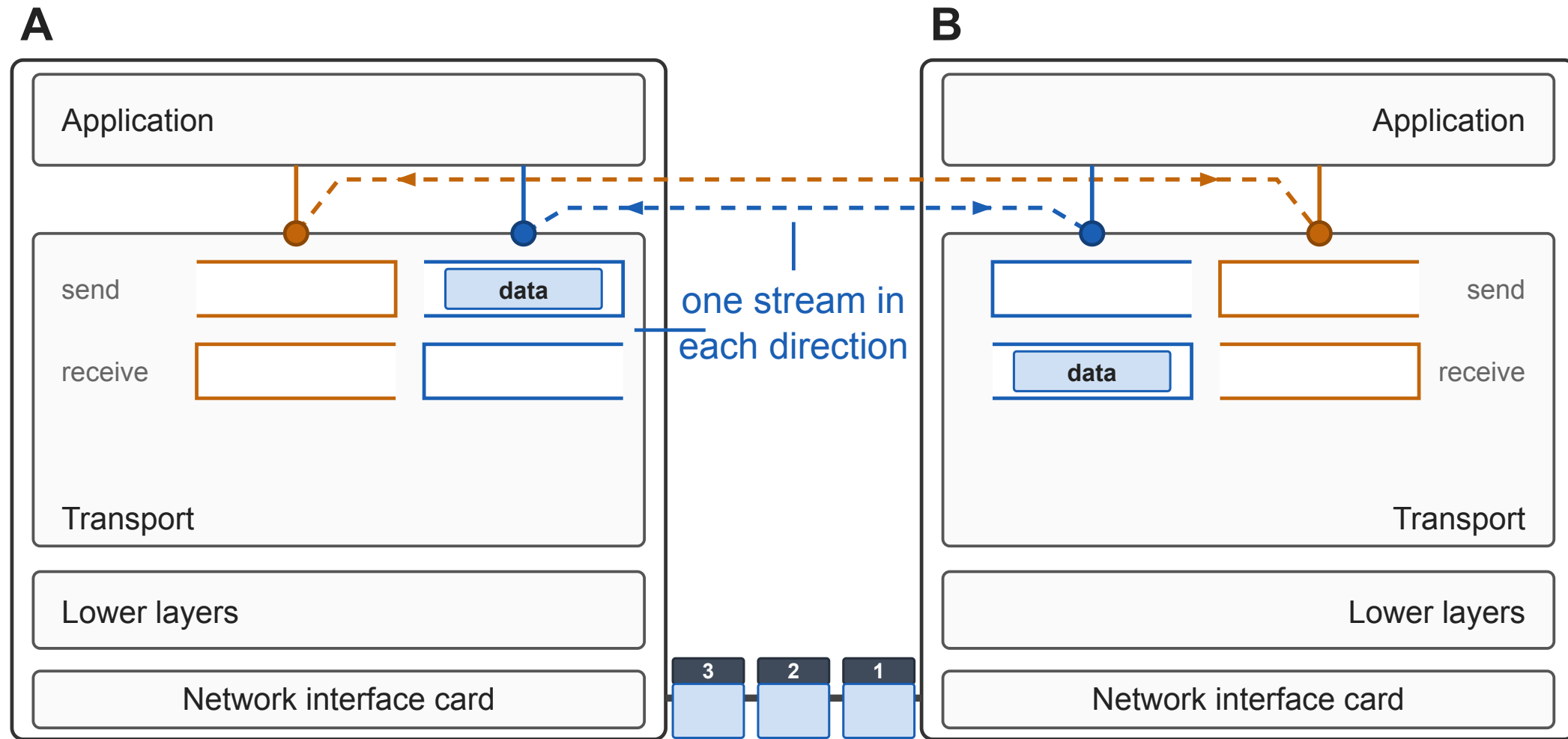


# **The transport layer: Dealing with bit errors and loss**

CS 456/656, Fall 2026

Mina Tahmasbi Arashloo

# Where we are



each packet carries its channel ID and its segment ID

# What does the transport layer have to take care of?

---

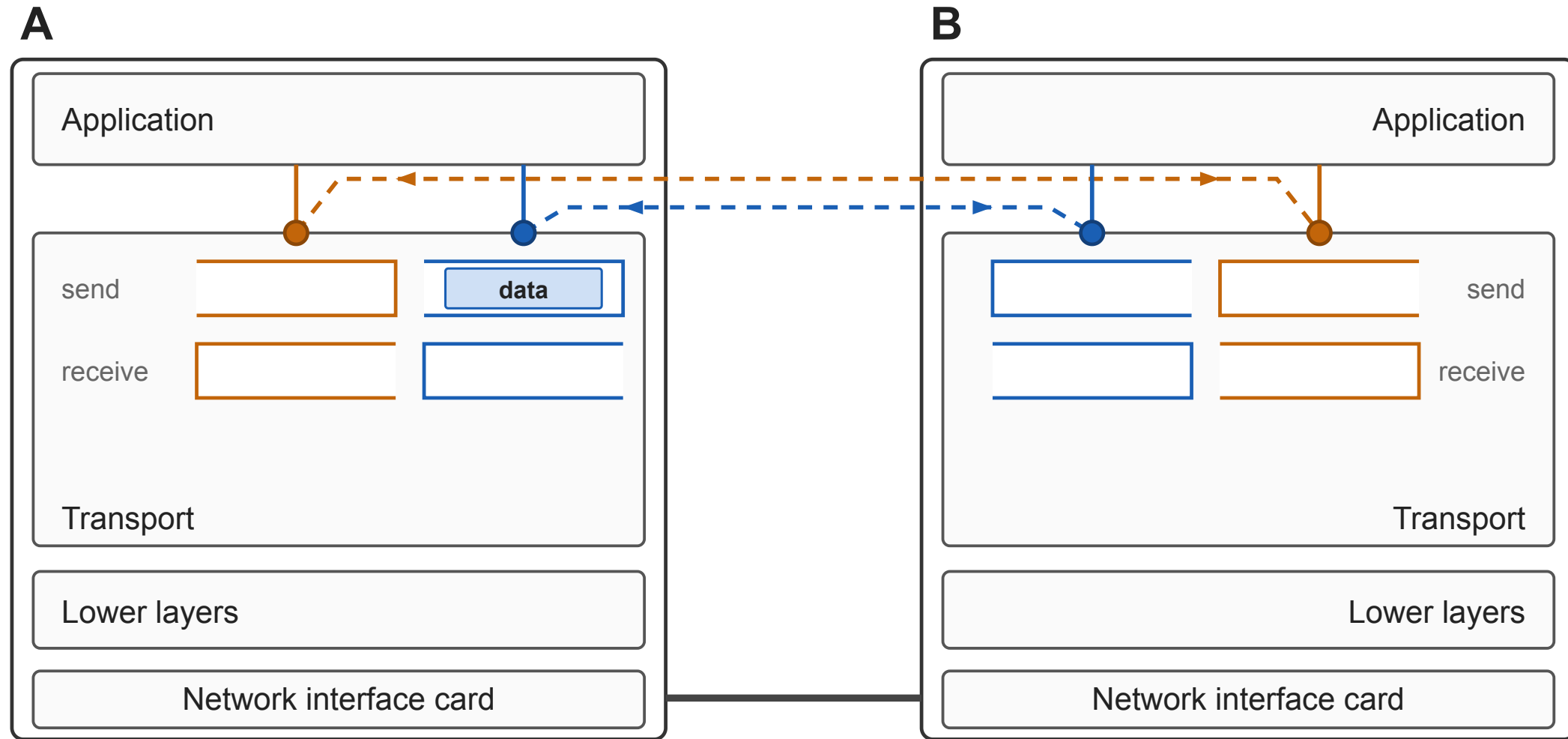
1. Get the data to the right application on the destination computer
2. Segmentation and reassembly
3. What to do when the lower layers lose, corrupt, or slow down packets

# What does the transport layer have to take care of?

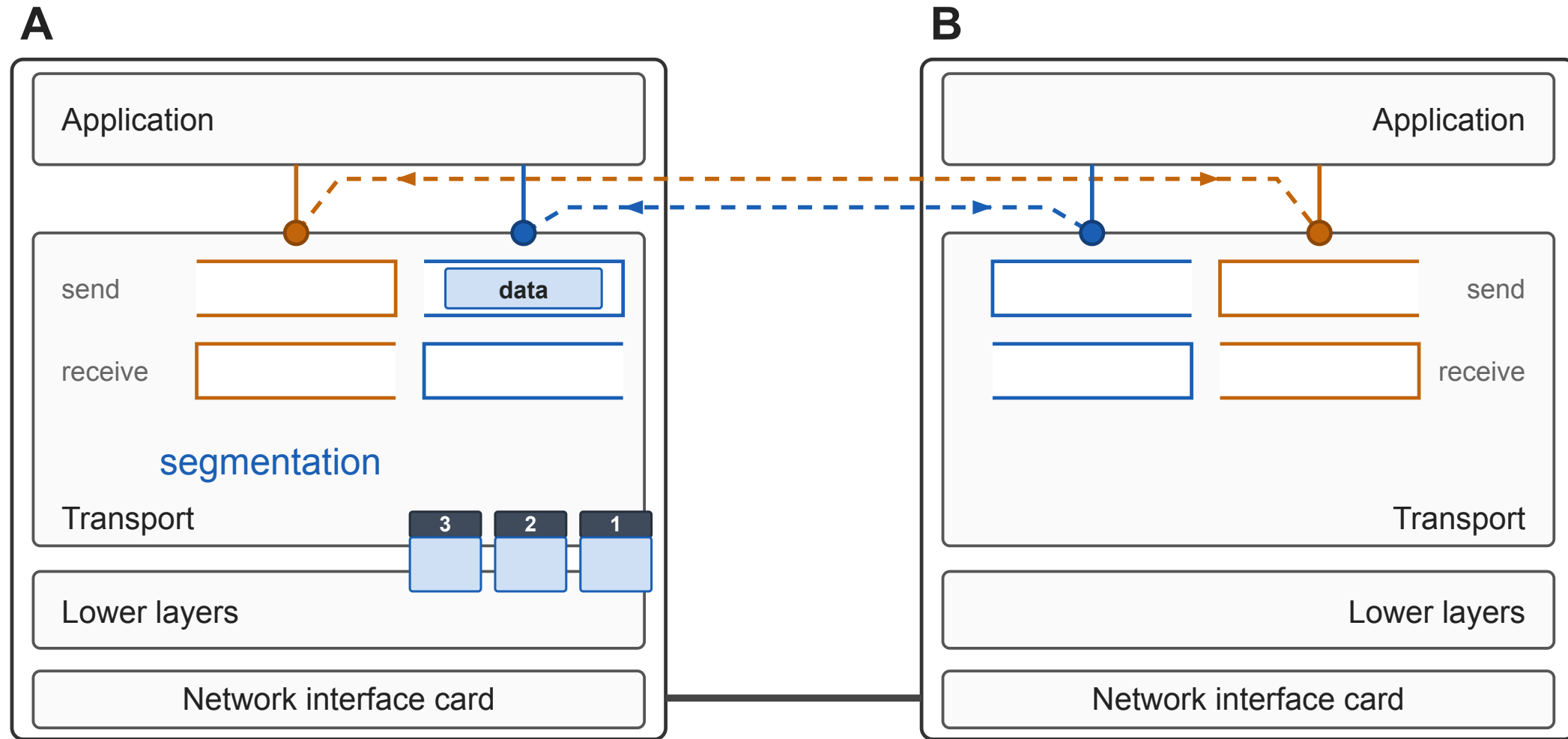
---

1. Get the data to the right application on the destination computer
2. Segmentation and reassembly
3. What to do when the lower layers lose, corrupt, or slow down packets

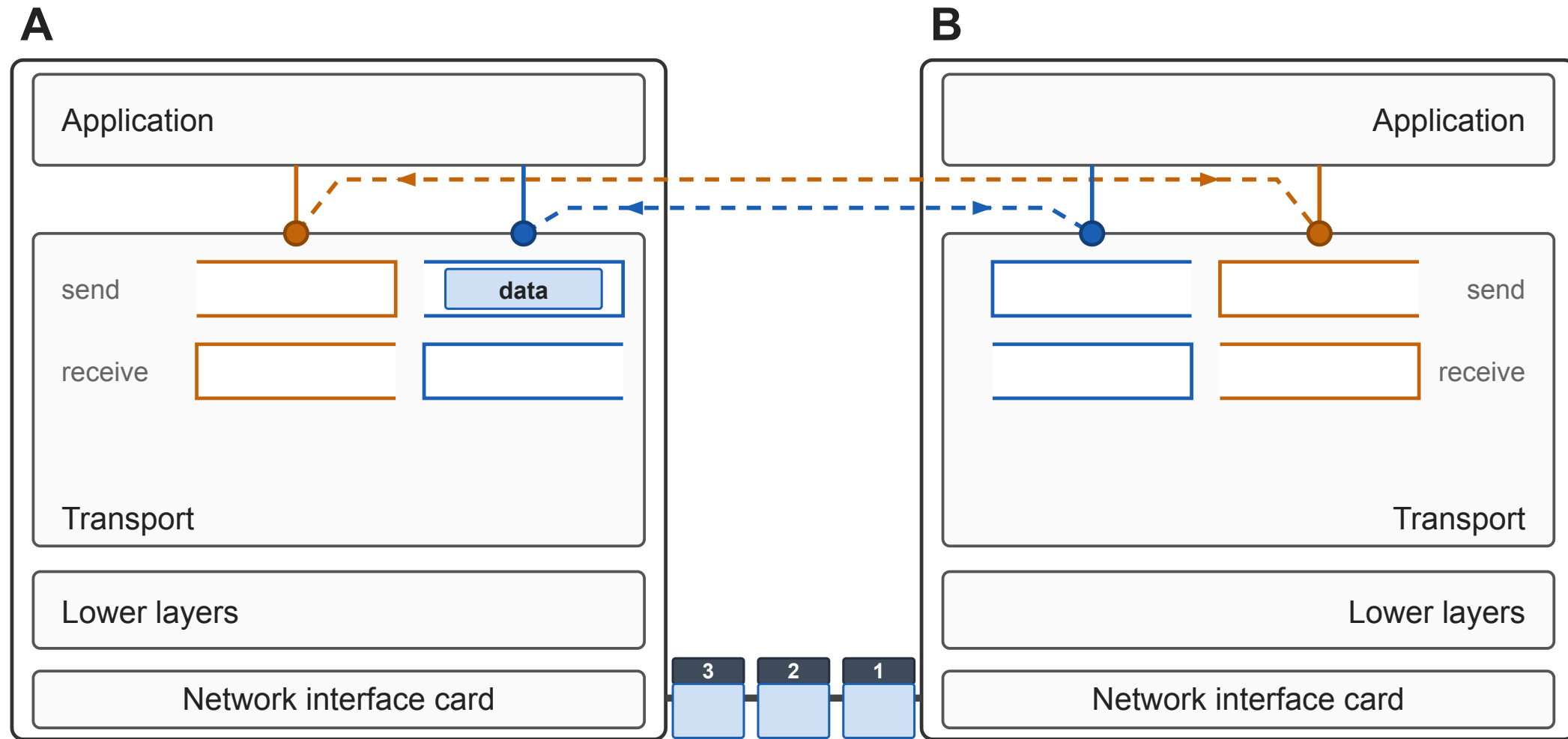
# What happens to the packets?



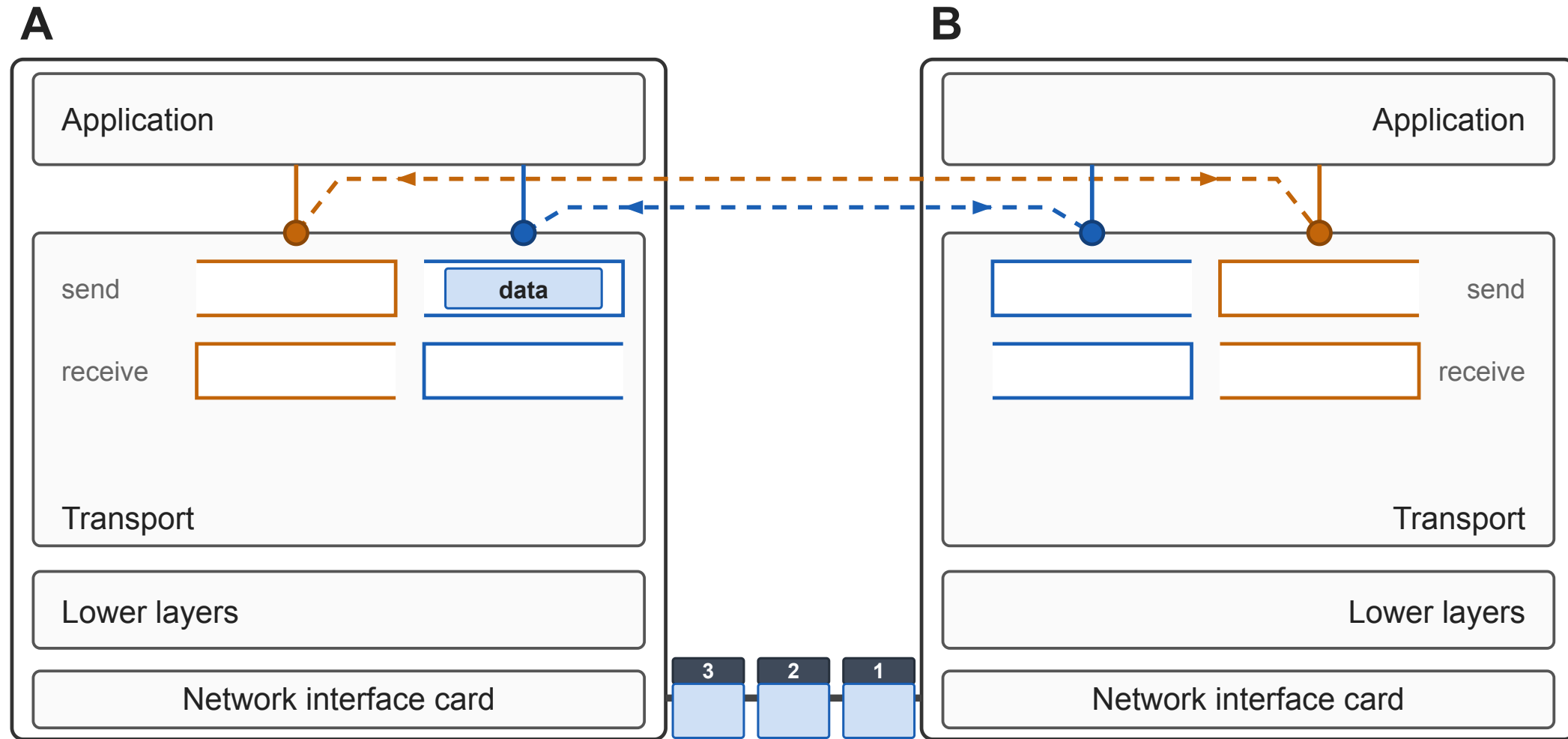
# What happens to the packets?



# What happens to the packets?



# What happens to the packets?



**The lower layers may corrupt, reorder, or drop these packets**

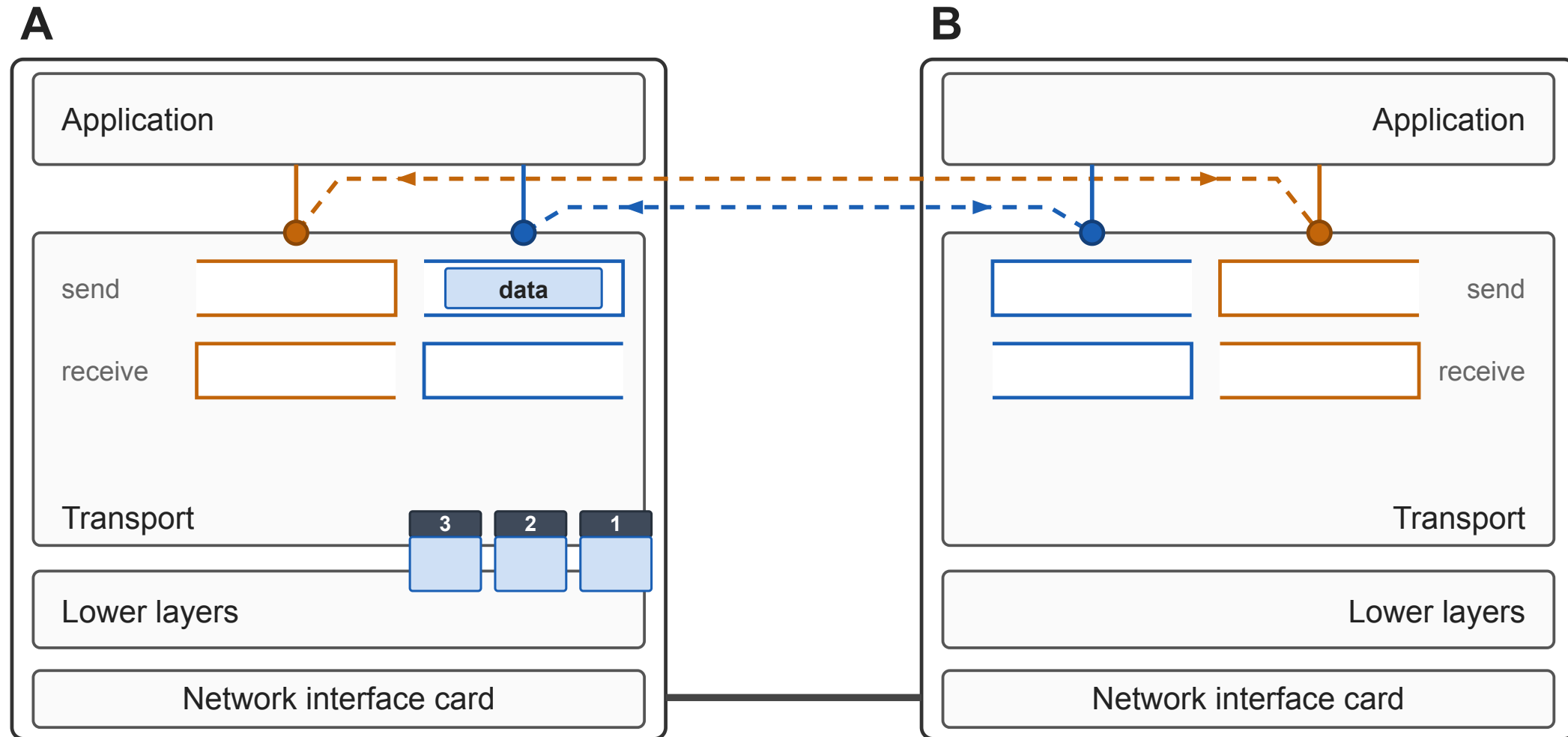


# First, one packet at a time

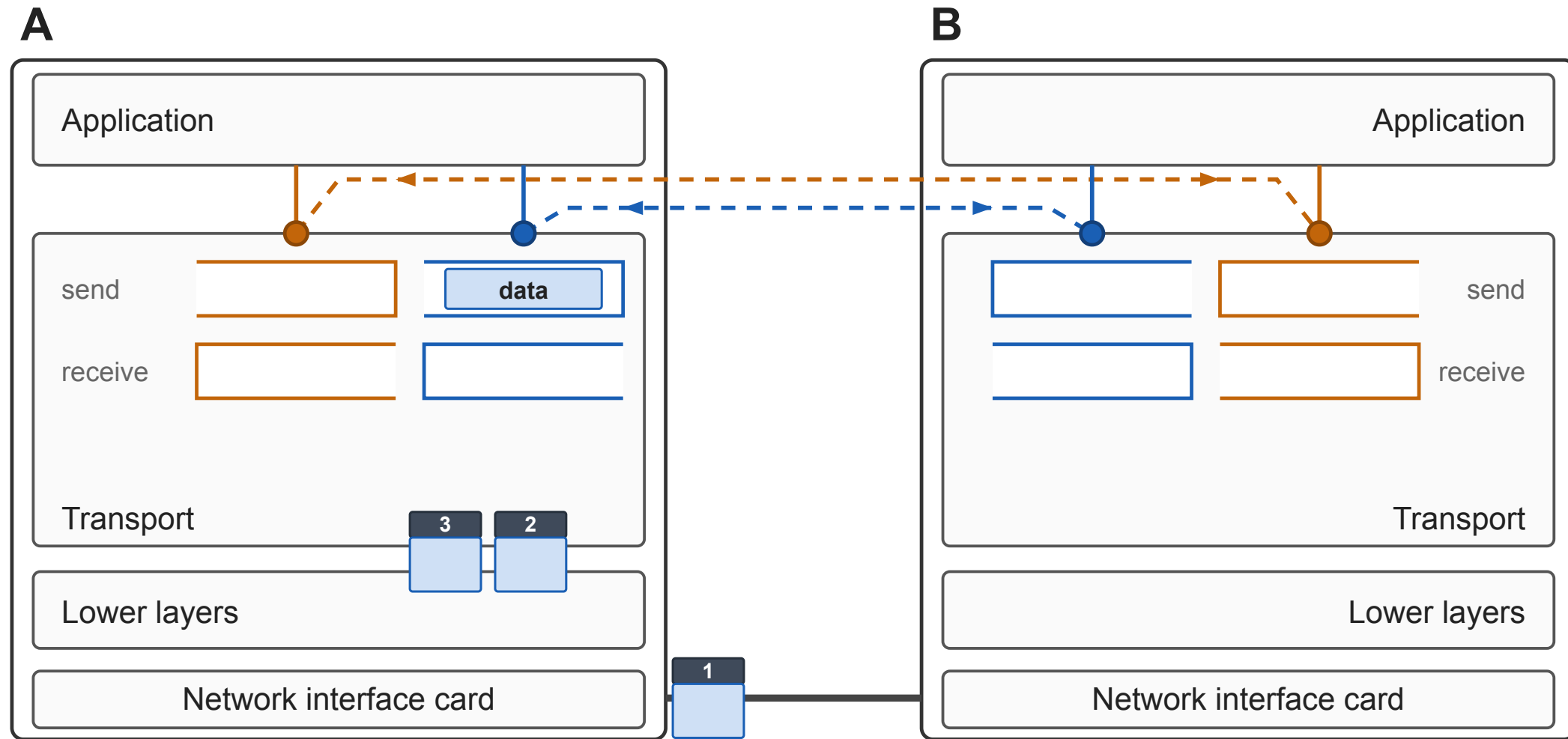
---

A's transport layer makes sure one packet has successfully been received at B, and only then moves on to the next one.

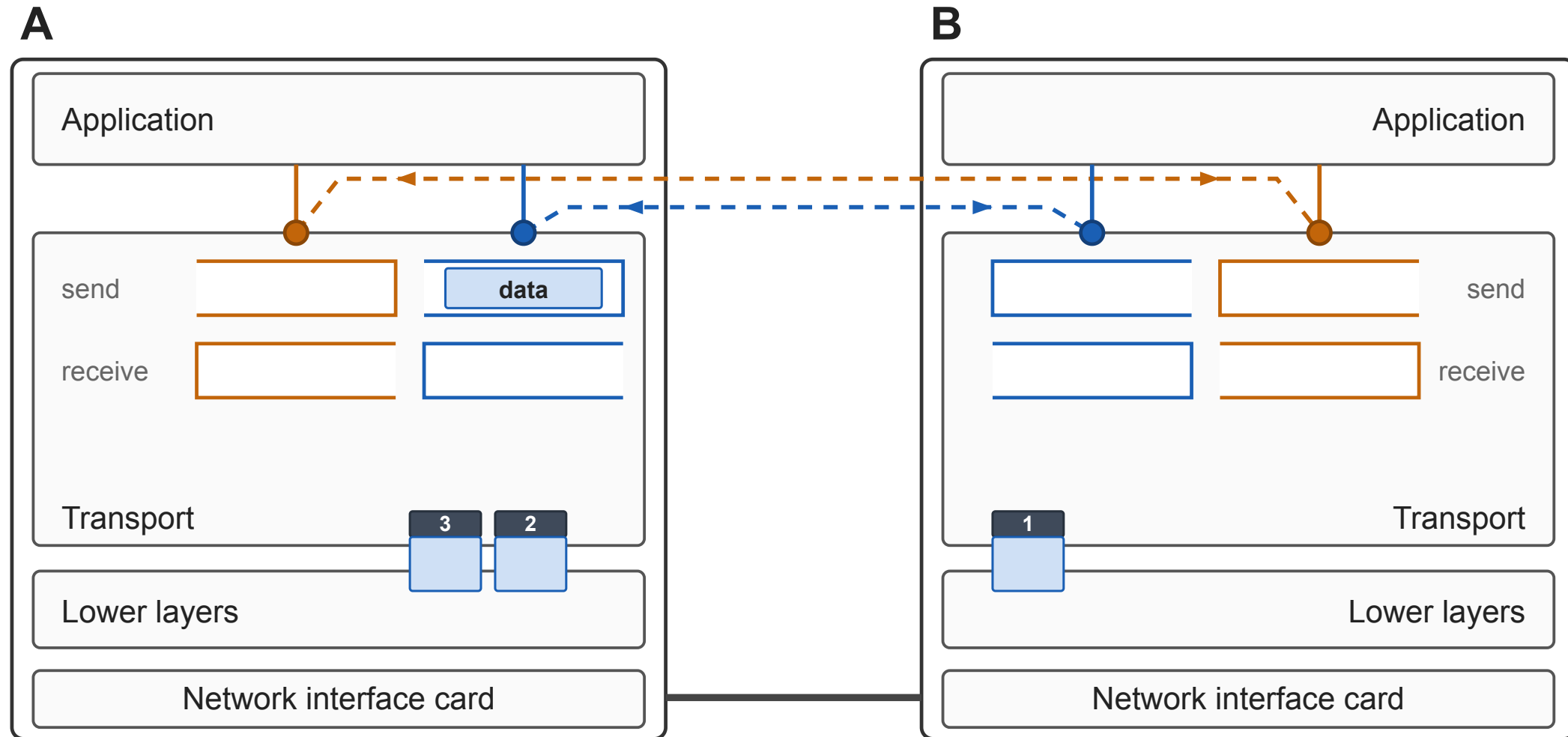
# One packet at a time



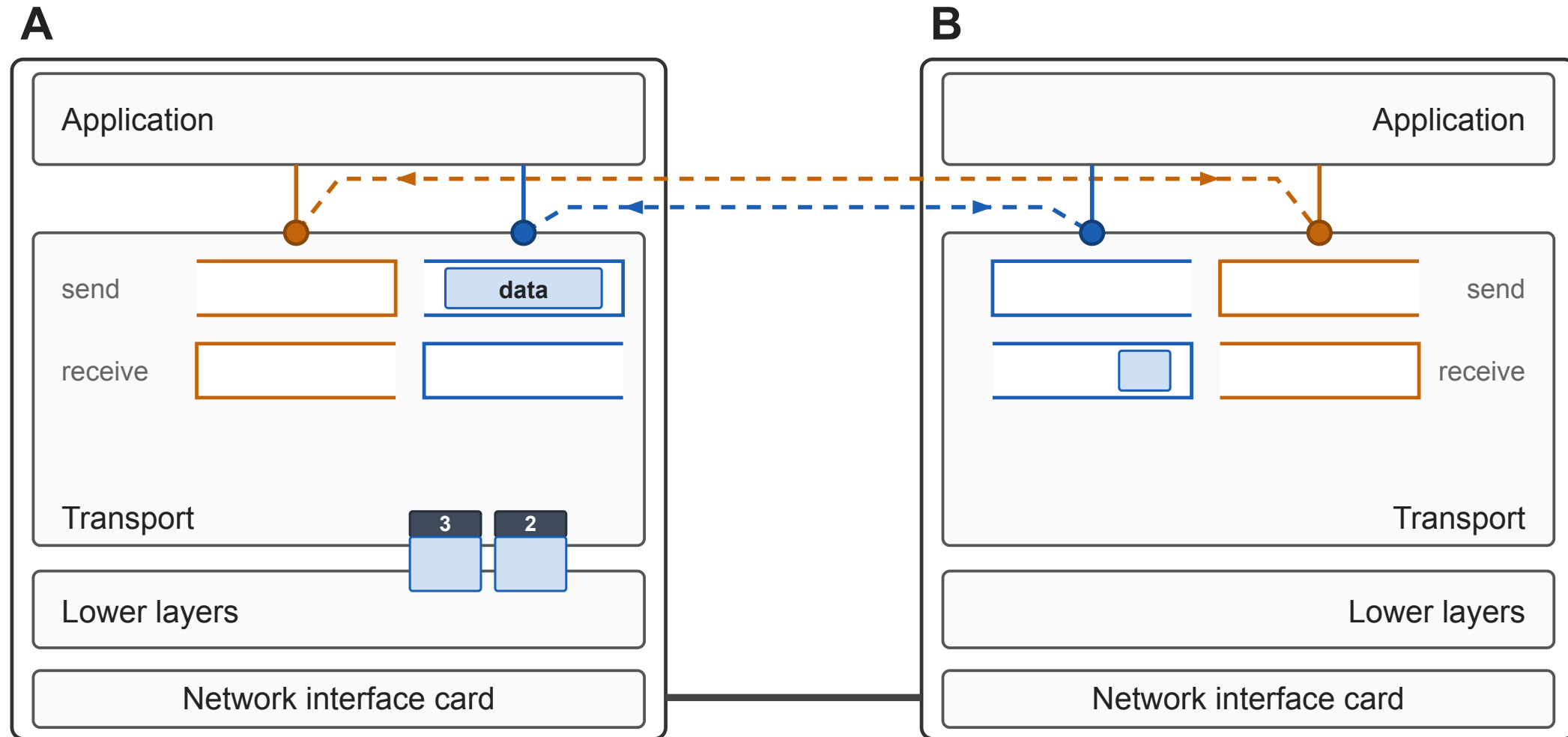
# One packet at a time



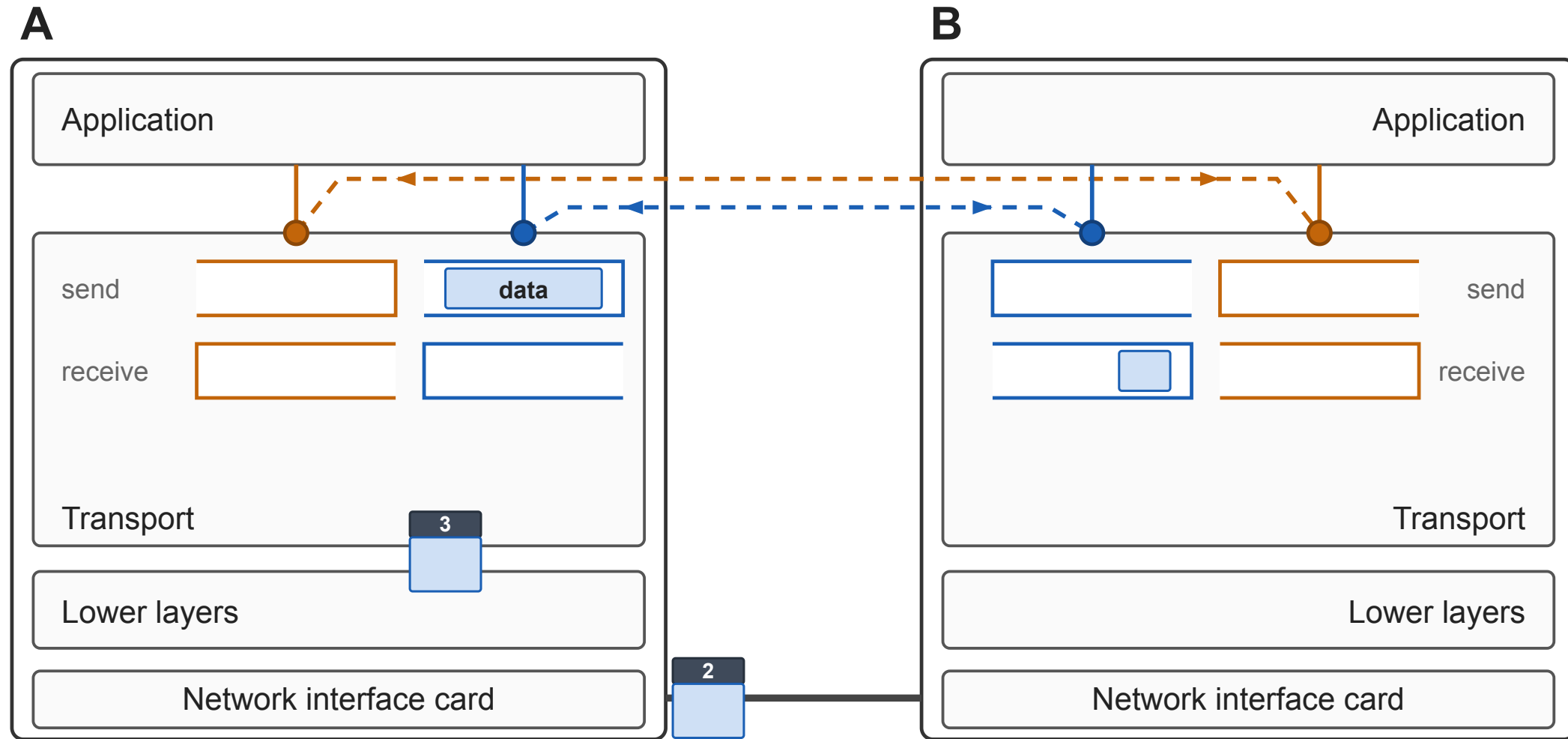
# One packet at a time



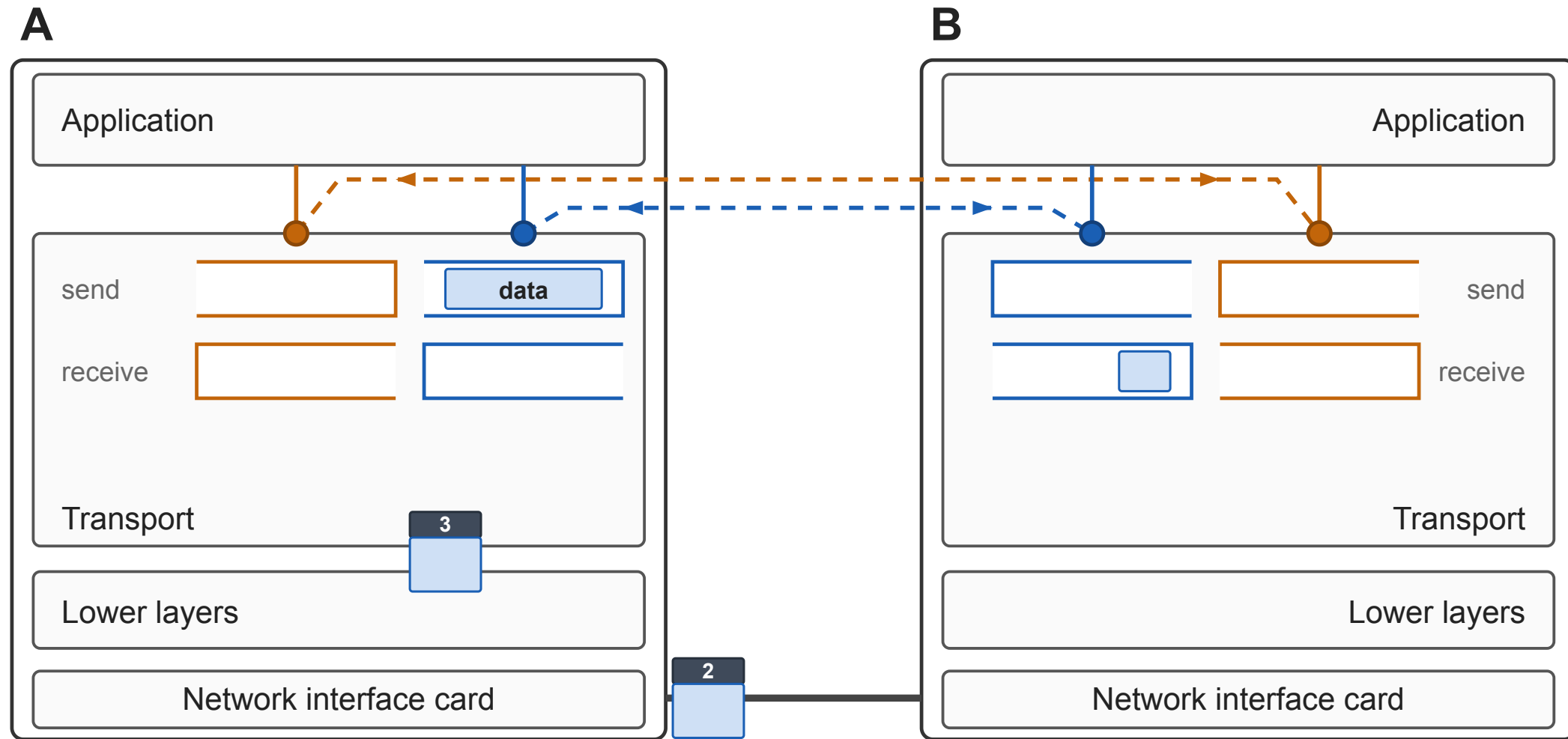
# One packet at a time



# One packet at a time



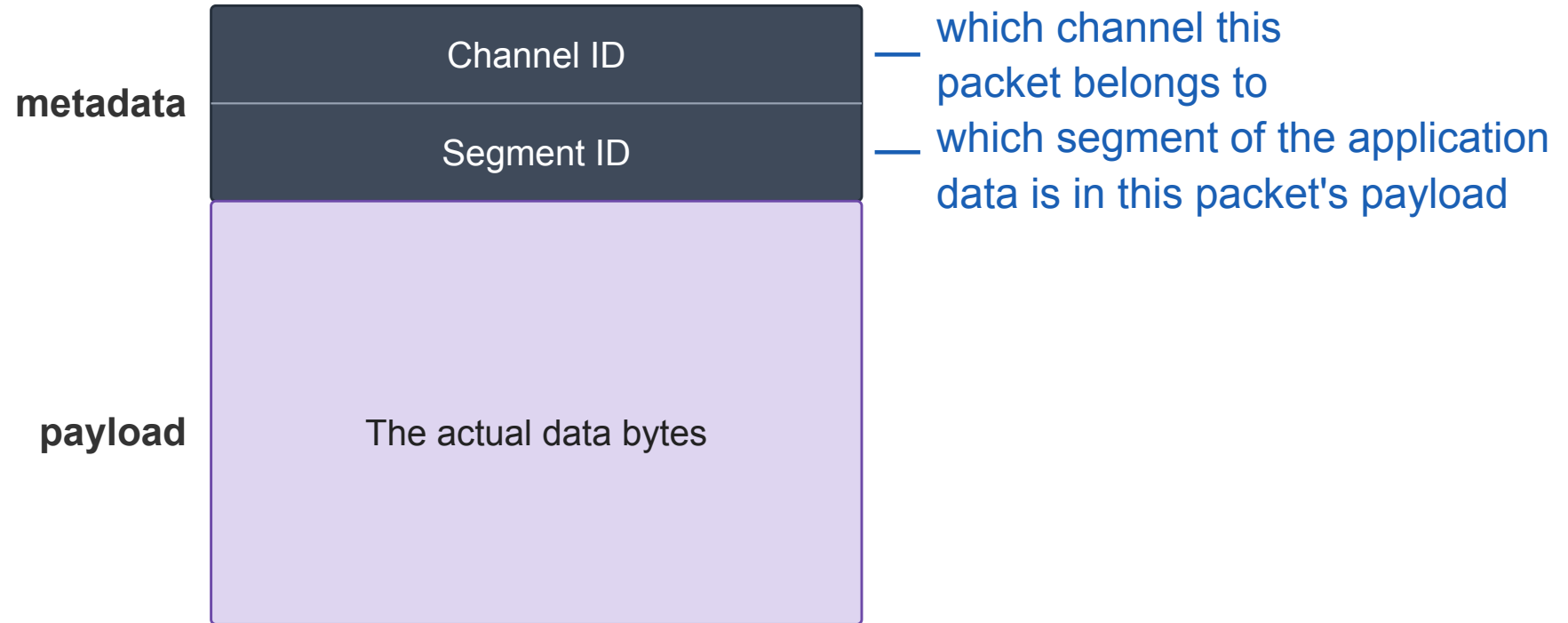
# One packet at a time



What should the protocol be, so that this one packet gets to B?

# Reminder: transport metadata so far

---





# Inside the transport layer, so far

---

*This is pseudocode. Real implementations optimize quite a bit and look different.*

`to_lower(pkt)` hands a packet to the lower layers. `from_lower()` blocks until they provide a received packet.

(\*) If the segments are not all `max_payload_size` long, this does not quite work. We overlook that for now.

## Transport at A (sender)

```
buffer send_buf[];    // the data, one per channel
int next_id[];        // next segment ID, per channel

void net_send(ch, data, n) {
    send_buf[ch].append(data, n);
}

void send_loop(ch) {
    while (true) {
        seg = send_buf[ch].take(max_payload_size);
        pkt = add_header(seg, ch, next_id[ch]);
        next_id[ch] = next_id[ch] + 1;
        to_lower(pkt);
    }
}
```

## Transport at B (receiver)

```
buffer recv_buf[];    // the data, one per channel

void net_recv(ch, data, n) {
    recv_buf[ch].take(data, n);
}

void receive_loop() {
    while (true) {
        pkt = from_lower();
        seg = strip_header(pkt);
        offset = pkt.seg_id * max_payload_size; // (*)
        recv_buf[pkt.ch].write(offset, seg);
    }
}
```

# Inside the transport layer, so far

*This is pseudocode. Real implementations optimize quite a bit and look different.*

`to_lower(pkt)` hands a packet to the lower layers. `from_lower()` blocks until they provide a received packet.

(\*) If the segments are not all `max_payload_size` long, this does not quite work. We overlook that for now.

## Transport at A (sender)

```
buffer send_buf[];    // the data, one per channel
int next_id[];        // next segment ID, per channel

void net_send(ch, data, n) {
    send_buf[ch].append(data, n);
}

void send_loop(ch) {
    while (true) {
        seg = send_buf[ch].take(max_payload_size);
        pkt = add_header(seg, ch, next_id[ch]);
        next_id[ch] = next_id[ch] + 1;
        to_lower(pkt);
    }
}
```

## Transport at B (receiver)

```
buffer recv_buf[];    // the data, one per channel

void net_recv(ch, data, n) {
    recv_buf[ch].take(data, n);
}

void receive_loop() {
    while (true) {
        pkt = from_lower();
        seg = strip_header(pkt);
        offset = pkt.seg_id * max_payload_size; // (*)
        recv_buf[pkt.ch].write(offset, seg);
    }
}
```

If the link is perfect (no corruption or loss) and B has infinite memory, this will be fine.

# Dealing with bit corruption

---

Every packet still arrives, in the order it was sent. But some of its bits may be flipped on the way.

# Dealing with bit corruption

---

Every packet still arrives, in the order it was sent. But some of its bits may be flipped on the way.

How can an endpoint tell that a packet it received is corrupted?

# Checksum

---

**A sends**

| channel ID: 1  |   |
|----------------|---|
| segment ID: 20 |   |
| 5              | 6 |

# Checksum

---

**A sends**

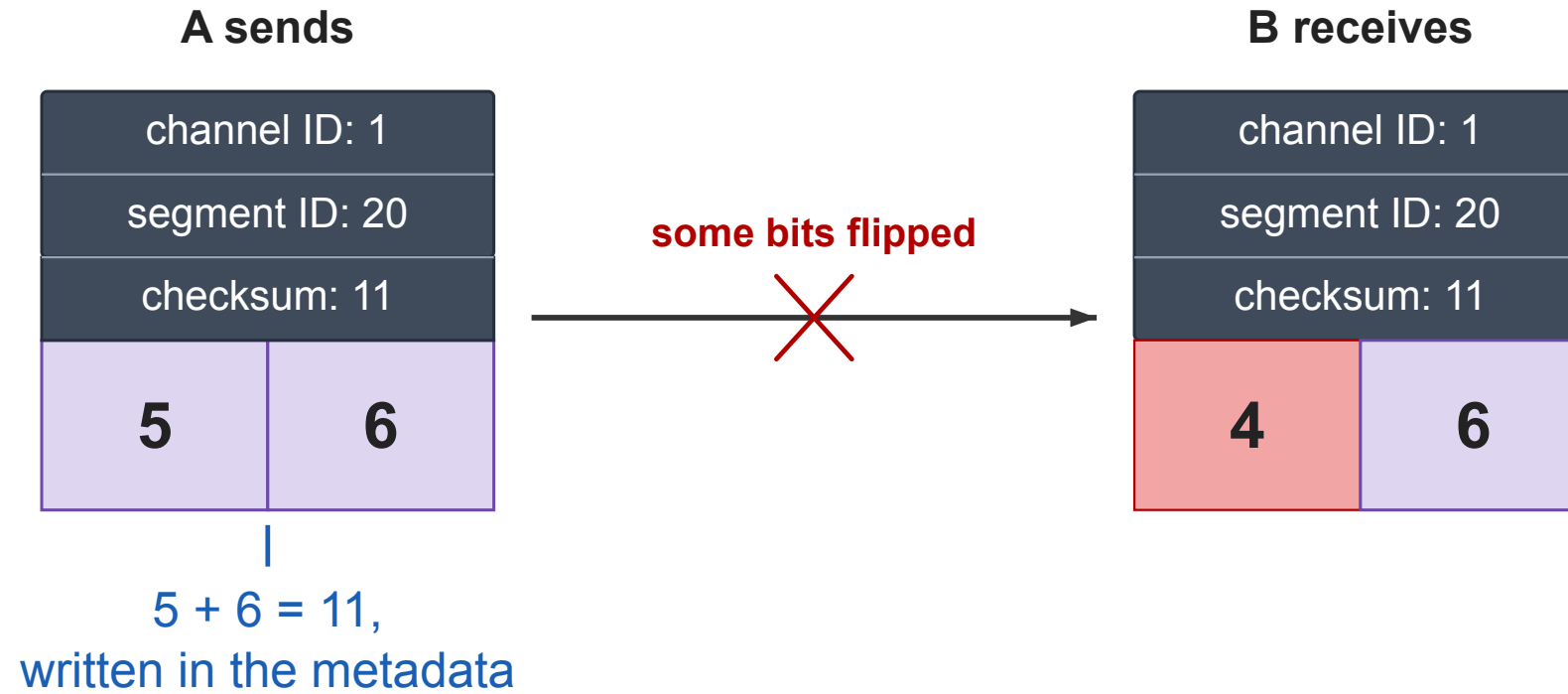
|                |   |
|----------------|---|
| channel ID: 1  |   |
| segment ID: 20 |   |
| checksum: 11   |   |
| 5              | 6 |

|

$5 + 6 = 11$ ,  
written in the metadata

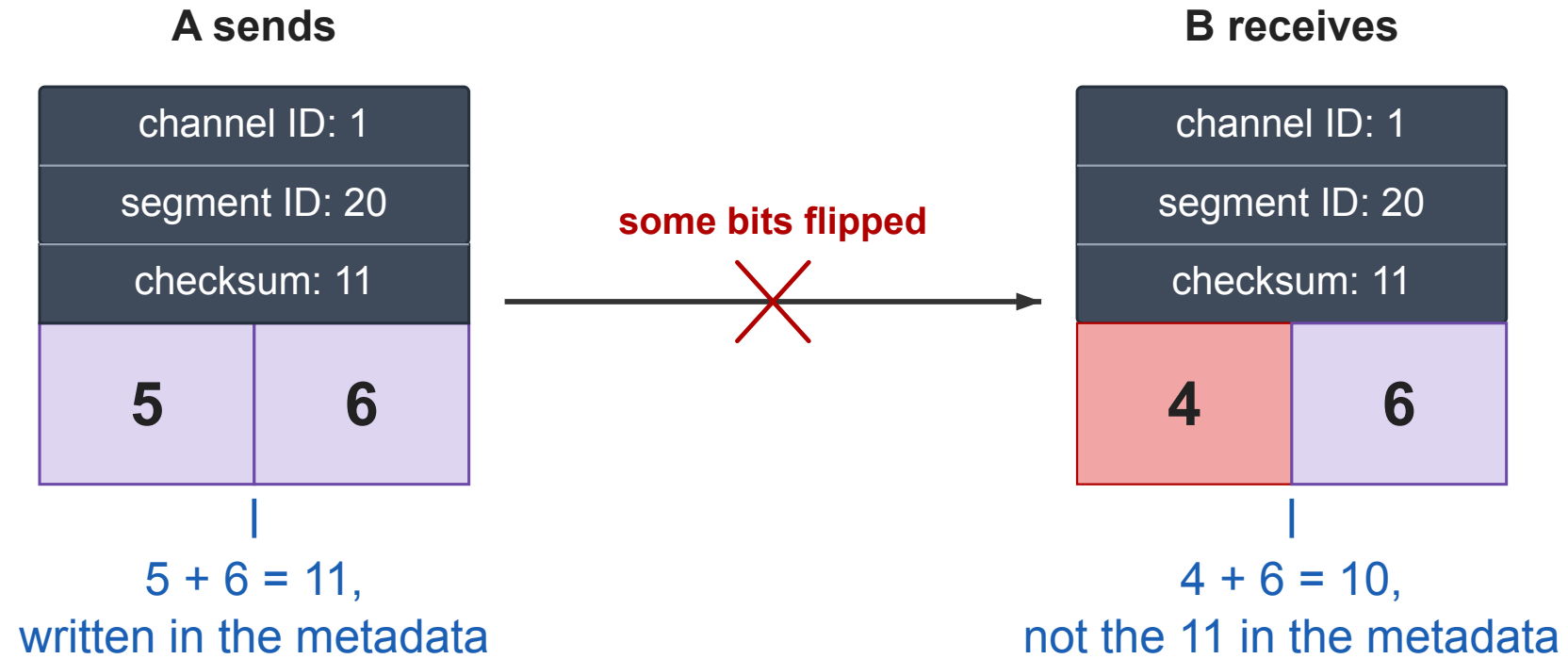
# Checksum

---



# Checksum

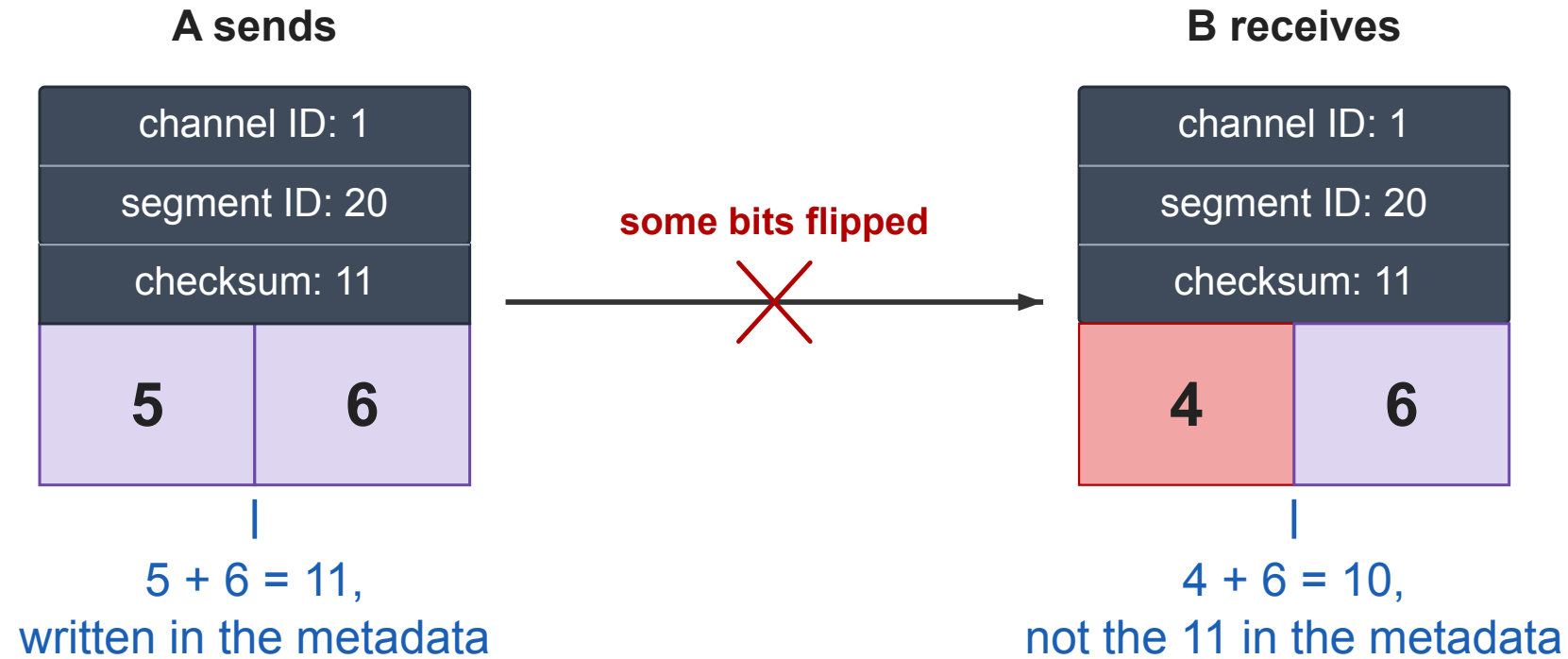
---





# Checksum

---



**B knows some bits flipped on the way (not which ones, though)**

# The Internet checksum

---

- **Sender:**
  - treat the metadata and payload as a sequence of 16-bit integers
  - add them all up, with one's complement addition
  - put the complement of the sum in the metadata, as the checksum
- **Receiver:** compute the same thing over what arrived, and compare it with the checksum in the metadata
  - not equal: an error is detected
  - equal: no error is detected (it does not mean there was no error)

# Internet checksum: an example

---

two 16-bit integers

$$\begin{array}{r} 0\ 1\ 1\ 0\ 0\ 1\ 1\ 0\ 0\ 1\ 1\ 0\ 0\ 1\ 1\ 0 \\ +\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1\ 0\ 1 \\ \hline \end{array}$$

# Internet checksum: an example

---

two 16-bit integers

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|   | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| + | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |

---

their sum

|  |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|--|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|  | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
|--|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# Internet checksum: an example

---

two 16-bit integers

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|   | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| + | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |

---

their sum

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

checksum: flip every bit

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

|

this goes in the packet's metadata

# Internet checksum does not always detect bit errors

---

**A sends**

two 16-bit integers

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|   | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| + | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |

their sum

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

checksum: flip every bit

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

# Internet checksum does not always detect bit errors

---

## A sends

two 16-bit integers

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|   | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| + | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |

---

their sum

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

checksum: flip every bit

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

## B receives

two bits flipped

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|   | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| + | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |

---

# Internet checksum does not always detect bit errors

---

## A sends

two 16-bit integers

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|   | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| + | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |

their sum

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

checksum: flip every bit

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

## B receives

two bits flipped

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|   | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| + | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |

the same sum

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

so the same checksum

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|



# Internet checksum does not always detect bit errors

---

## A sends

two 16-bit integers

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|   | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| + | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |

their sum

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

checksum: flip every bit

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

## B receives

two bits flipped

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|   | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
| + | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |

the same sum

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

so the same checksum

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

**Although the bits changed, the checksum did not change,  
so this corruption goes unnoticed**

# Who do you think should deal with unnoticed corruption?

(in-class discussion)

---

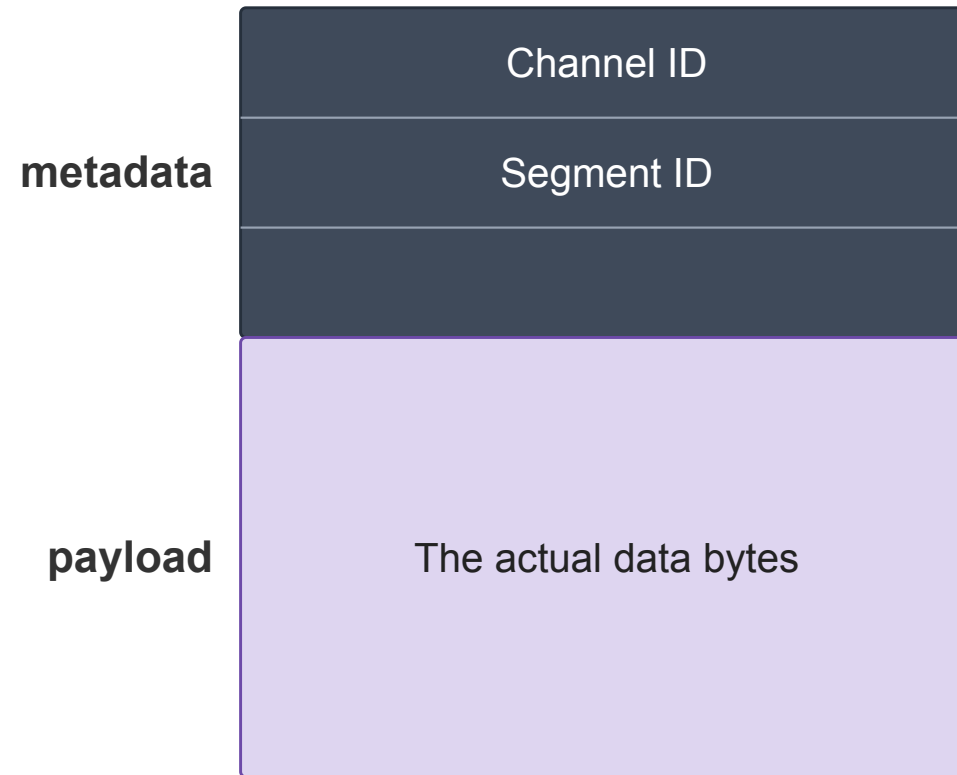
(in class discussion):

- The application should have stronger protection itself against data corruption?
- Catch them early on in the lower layers?

Reset

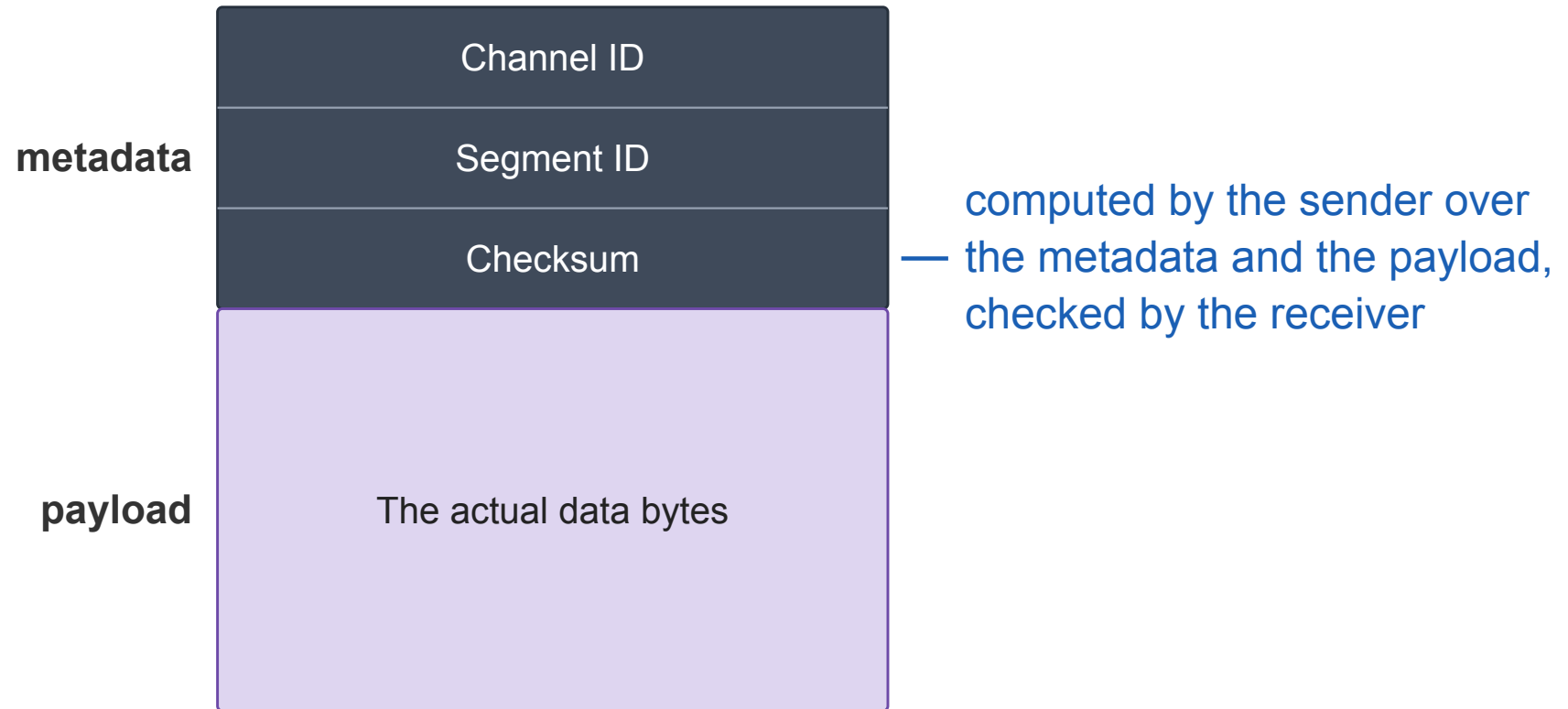
# The packet, with a checksum

---



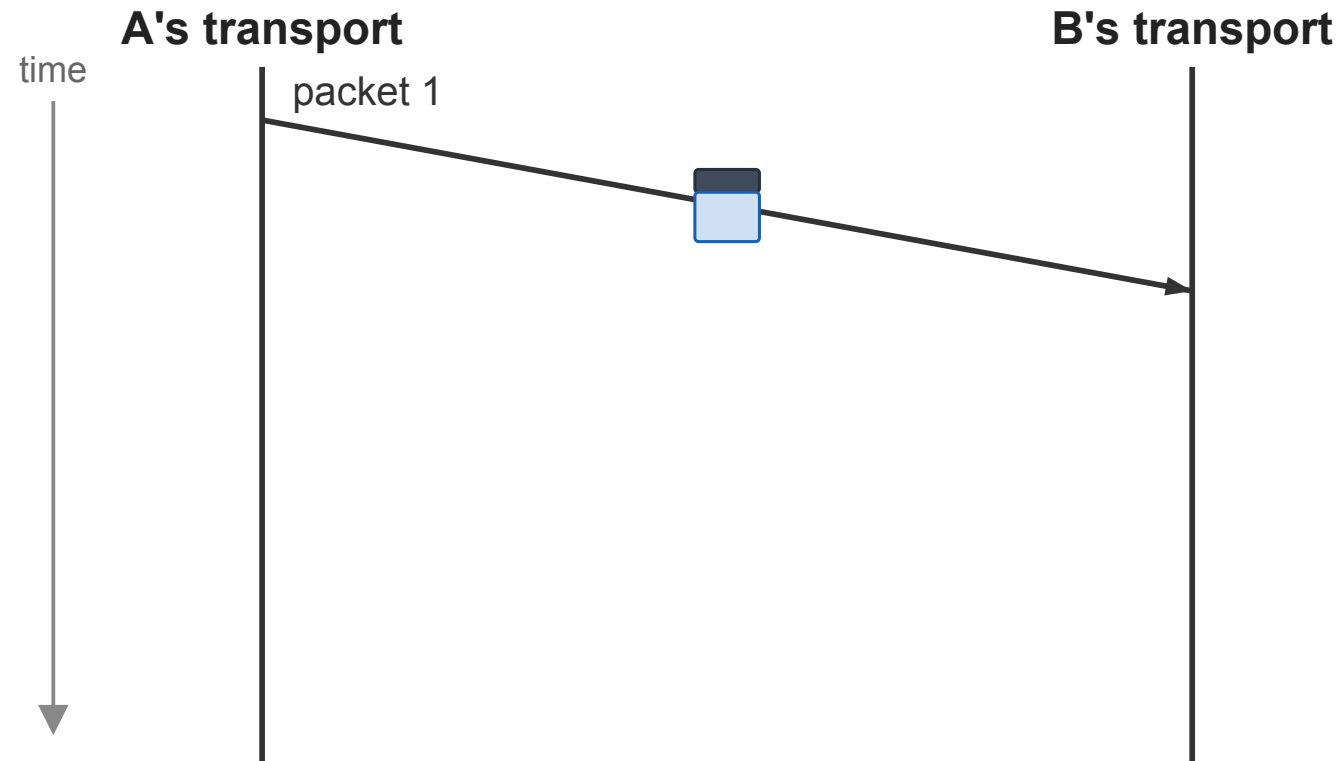
# The packet, with a checksum

---



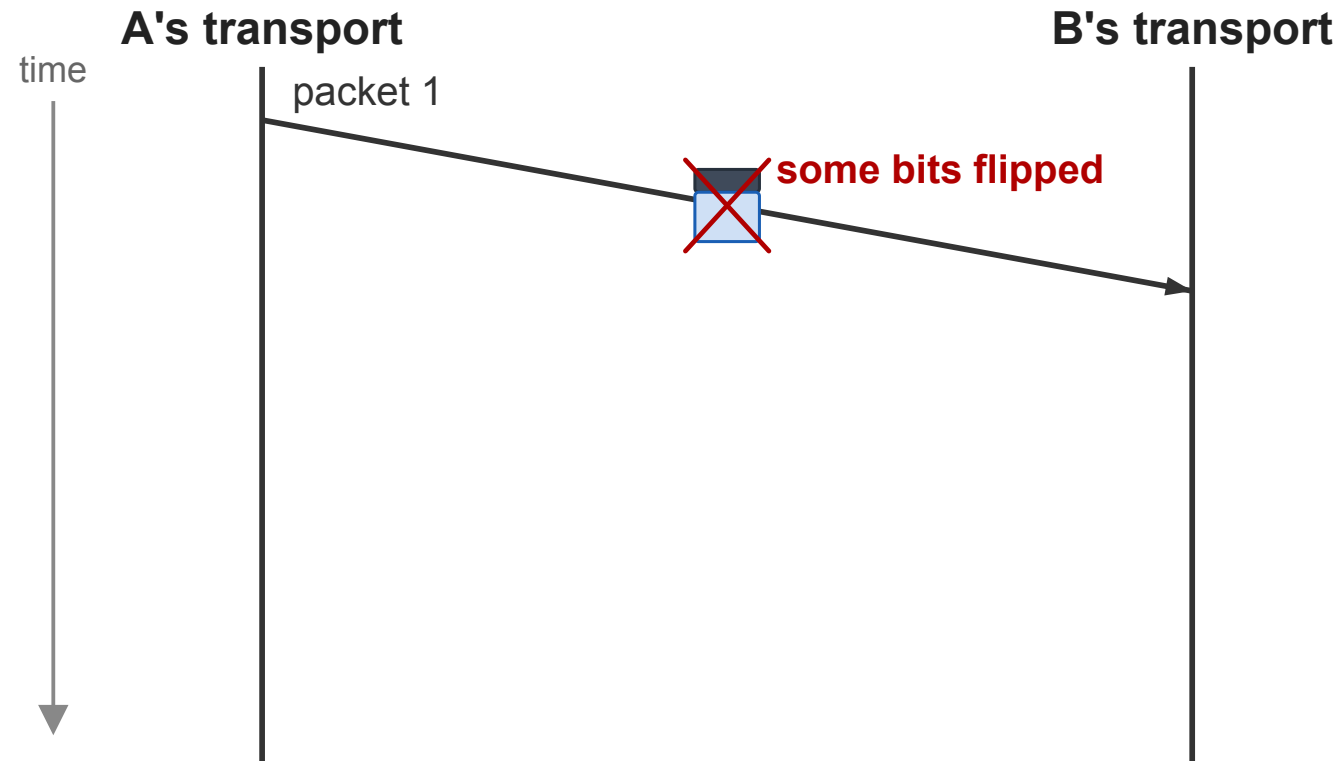
# When the data packet is corrupted

---



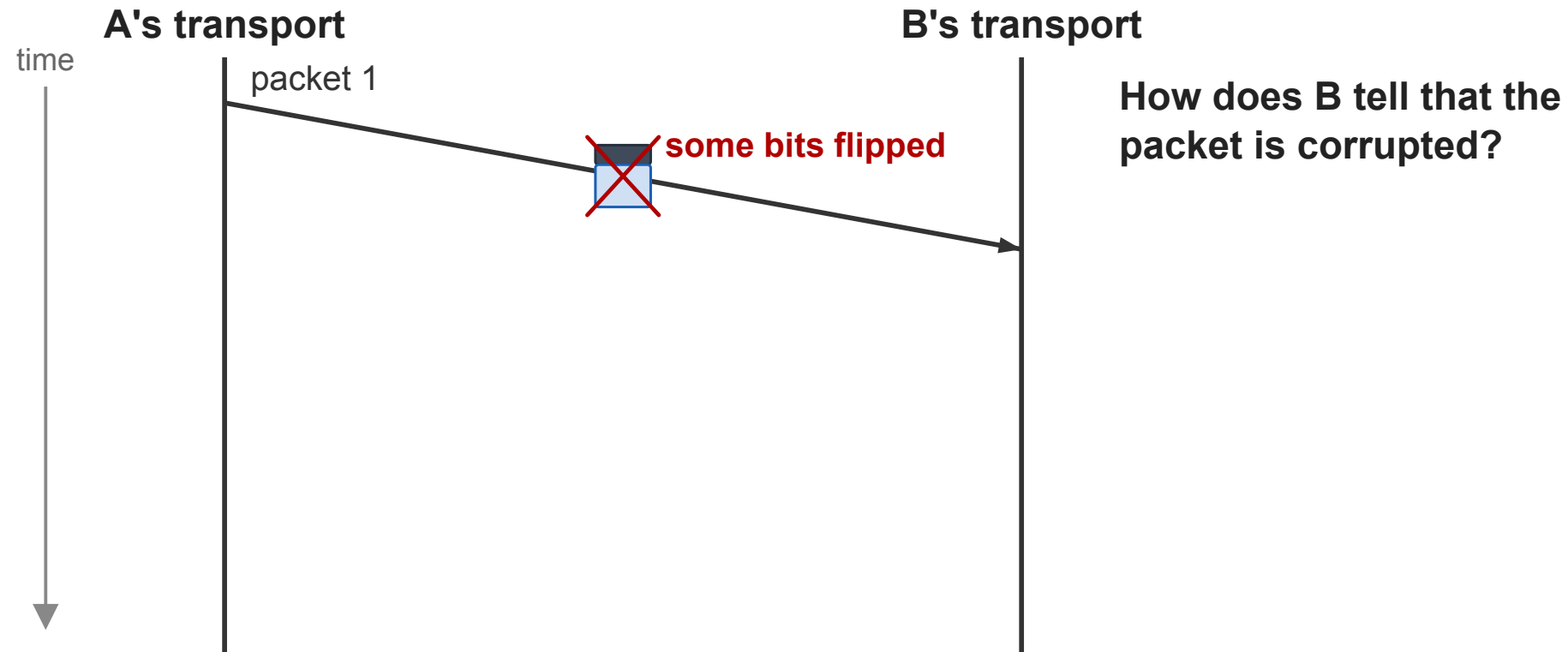
# When the data packet is corrupted

---



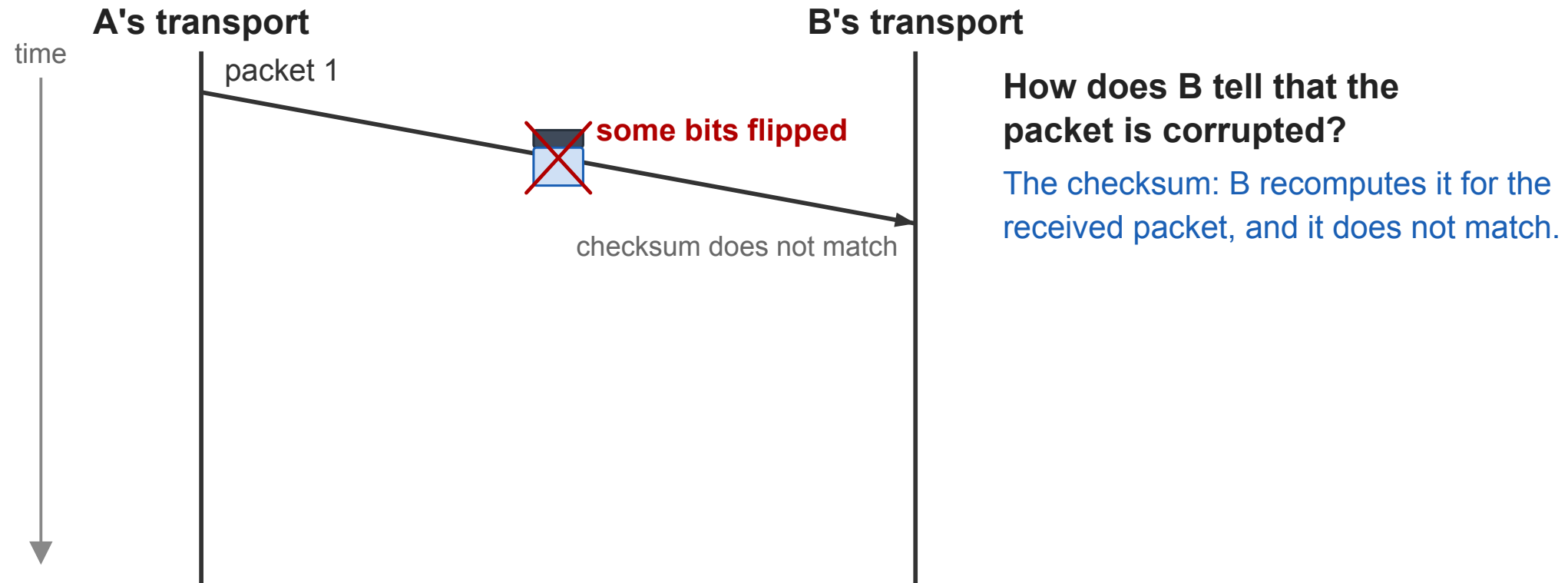
# When the data packet is corrupted

---



# When the data packet is corrupted

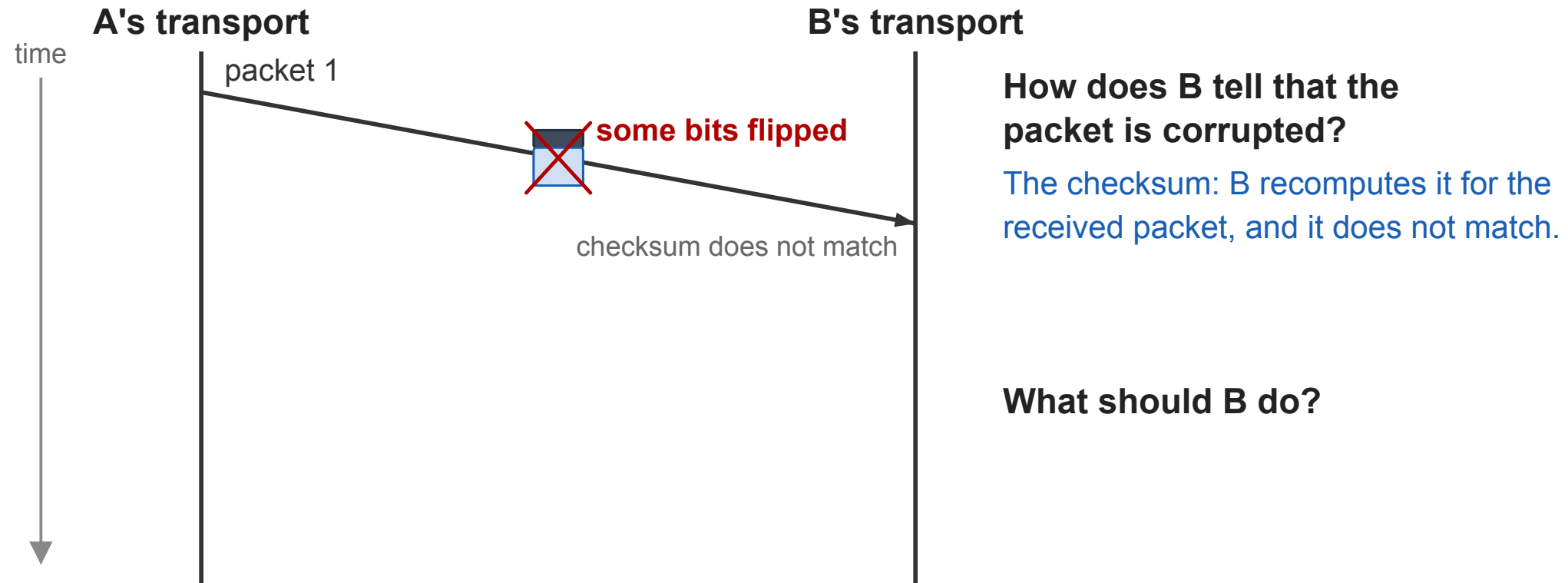
---





# When the data packet is corrupted

---



# Corrupted data packet - pseudocode (in-class solution)

## Transport at A (sender)

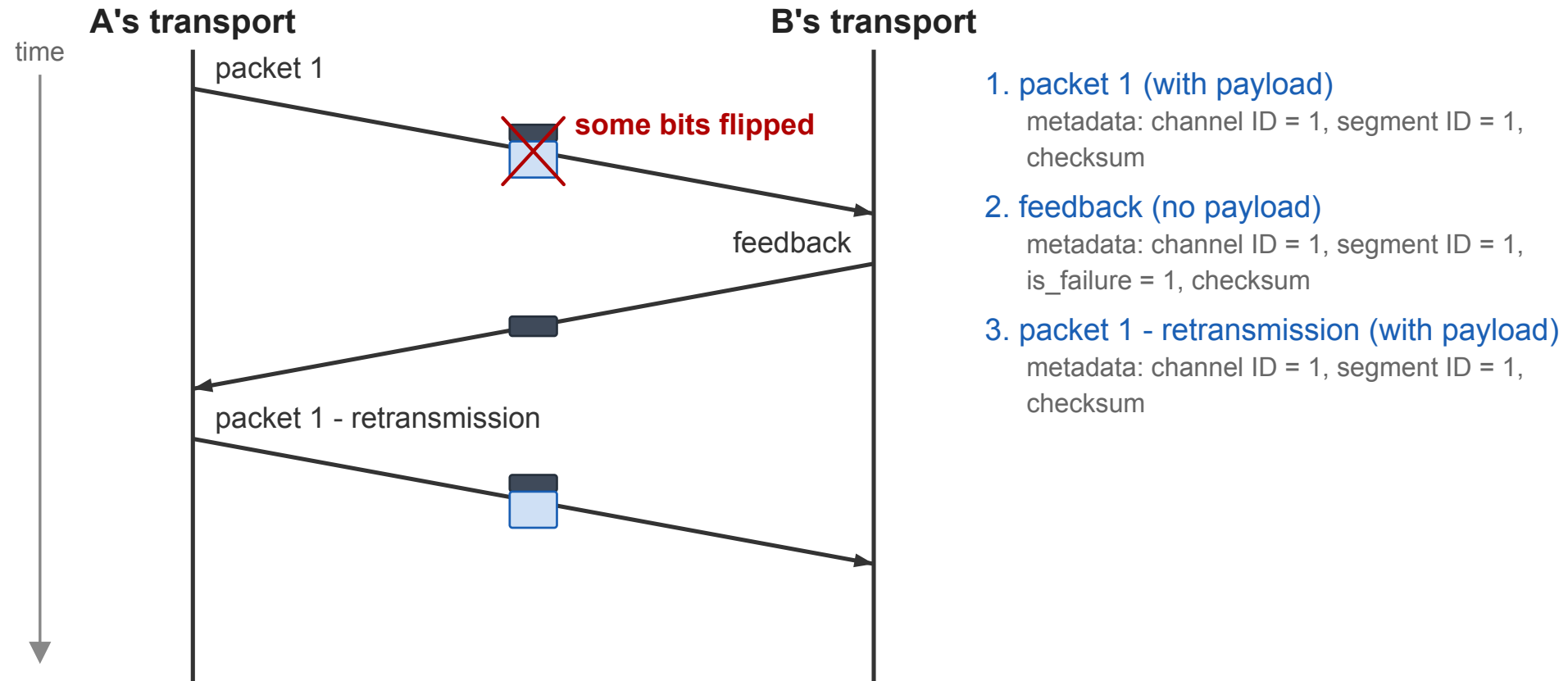
```
void send_loop(ch) {
    while (true) {
        seg = send_buf[ch].take(max_payload_size);
        pkt = add_header(seg, ch, next_id[ch]);
        // over the metadata and the payload
        pkt.checksum = compute_checksum(pkt);
        to_lower(pkt);

        while (true) {
            fb = from_lower();
            if (!corrupt(fb)) {
                if (fb.is_failure) {
                    to_lower(pkt);
                }
                else {
                    next_id[ch] = next_id[ch] + 1;
                    break;
                }
            }
            else {
                // TODO
            }
        }
    }
}
```

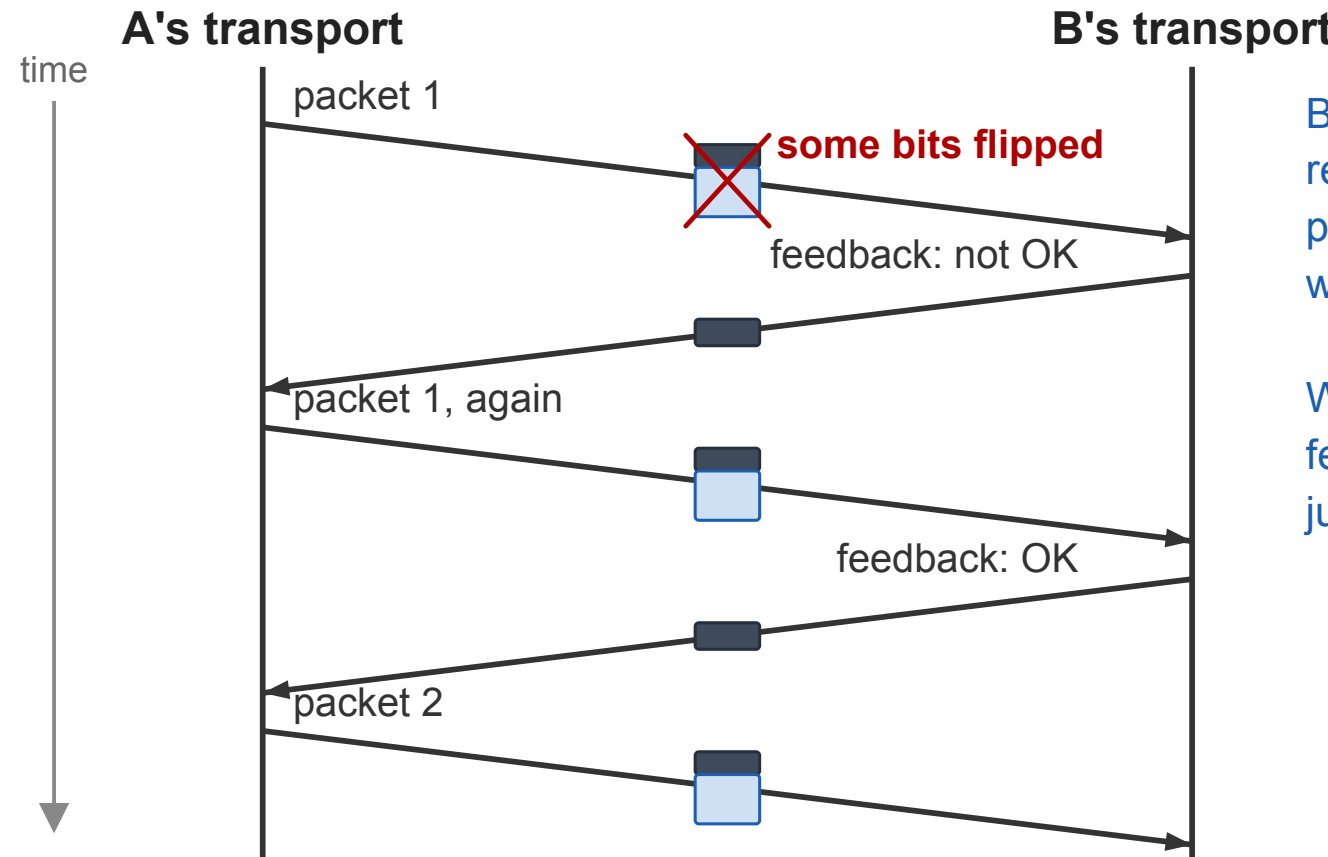
## Transport at B (receiver)

```
void receive_loop() {
    while (true) {
        pkt = from_lower();
        if (corrupt(pkt)) {
            // channel ID, segment ID, is_failure
            fb = feedback(pkt.ch, pkt.seg_id, 1);
            to_lower(fb);
        }
        else {
            seg = strip_header(pkt);
            offset = pkt.seg_id * max_payload_size;
            recv_buf[pkt.ch].write(offset, seg);
            fb = feedback(pkt.ch, pkt.seg_id, 0);
            to_lower(fb);
        }
    }
}
```

# Corrupted data packet - timeline (in-class solution)



# The receiver sends feedback for every packet

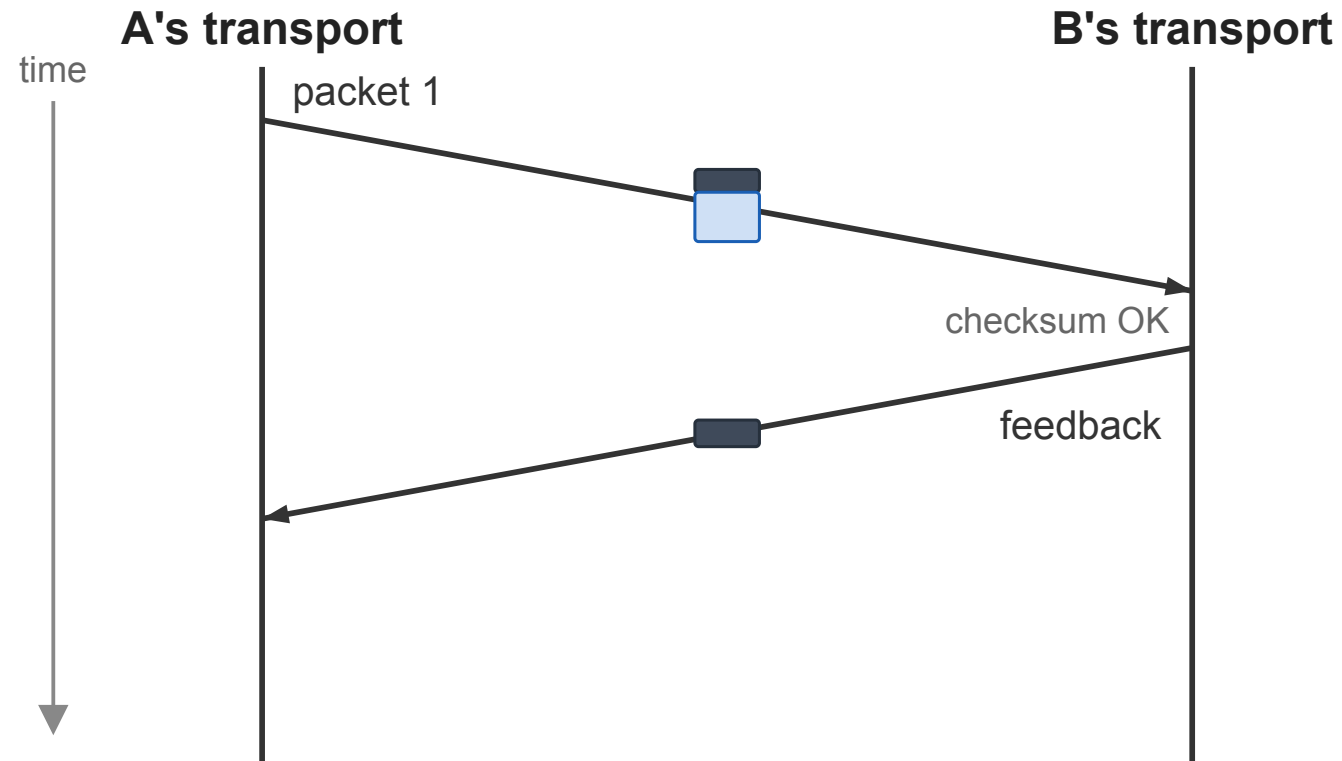


B sends feedback for every packet it receives: “positive” feedback when the packet is fine, “negative” feedback when the packet is corrupted.

What metadata is included in the feedback is a design choice. The one we just built in class is just one example.

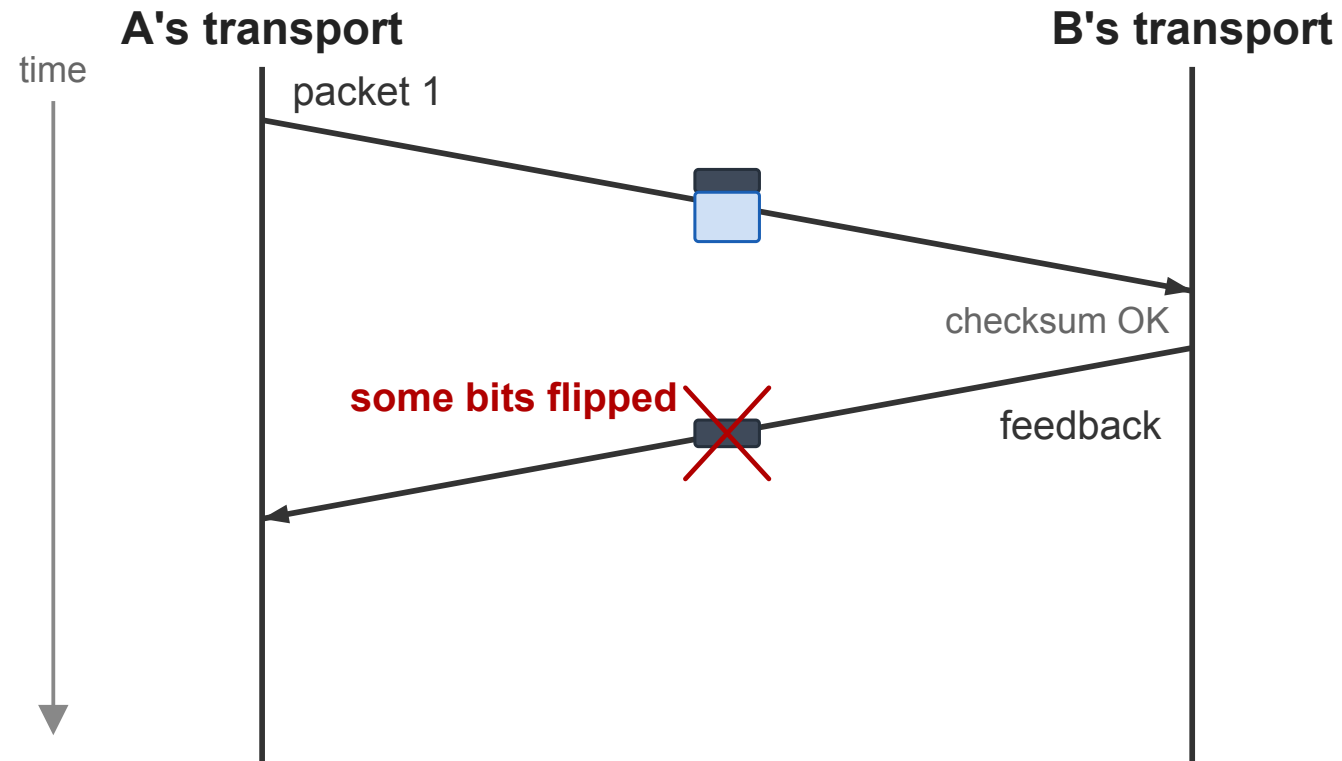
# When the feedback is corrupted

---



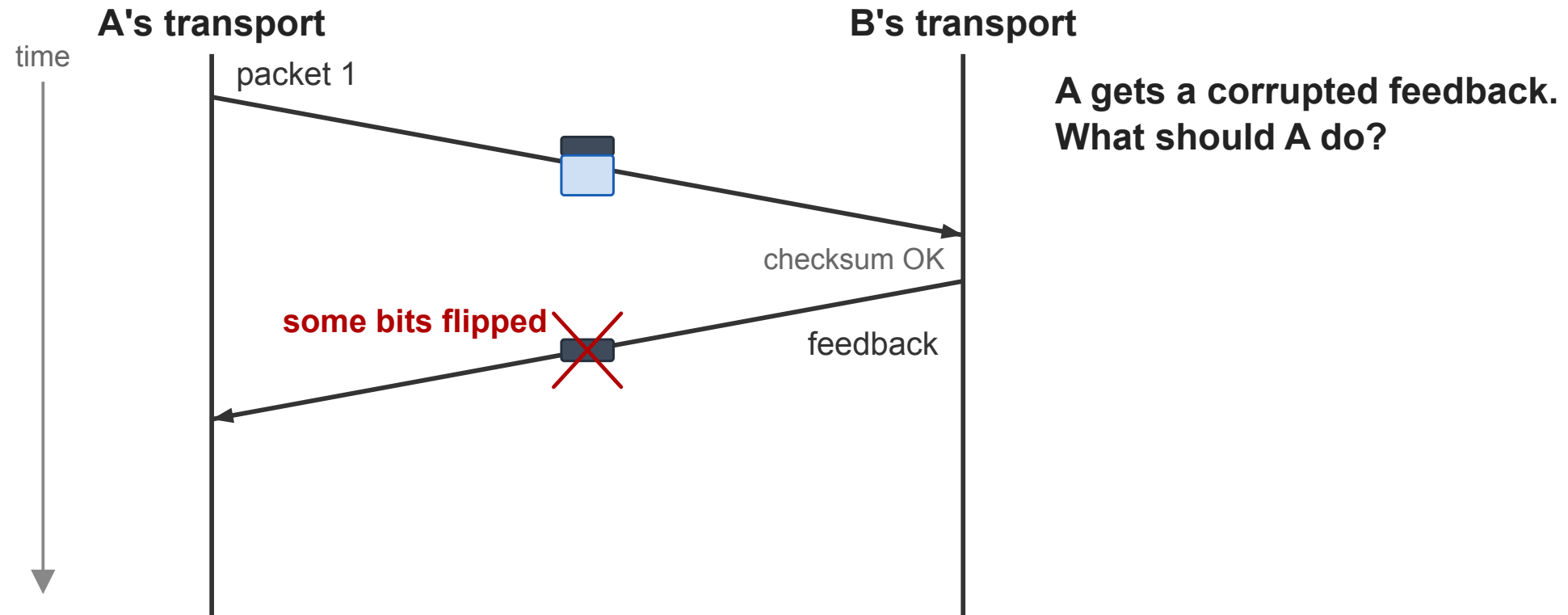
# When the feedback is corrupted

---



# When the feedback is corrupted

---



# Corrupted feedback - pseudocode (in-class solution)

## Transport at A (sender)

```
void send_loop(ch) {
    while (true) {
        seg = send_buf[ch].take(max_payload_size);
        pkt = add_header(seg, ch, next_id[ch]);
        // over the metadata and the payload
        pkt.checksum = compute_checksum(pkt);
        to_lower(pkt);

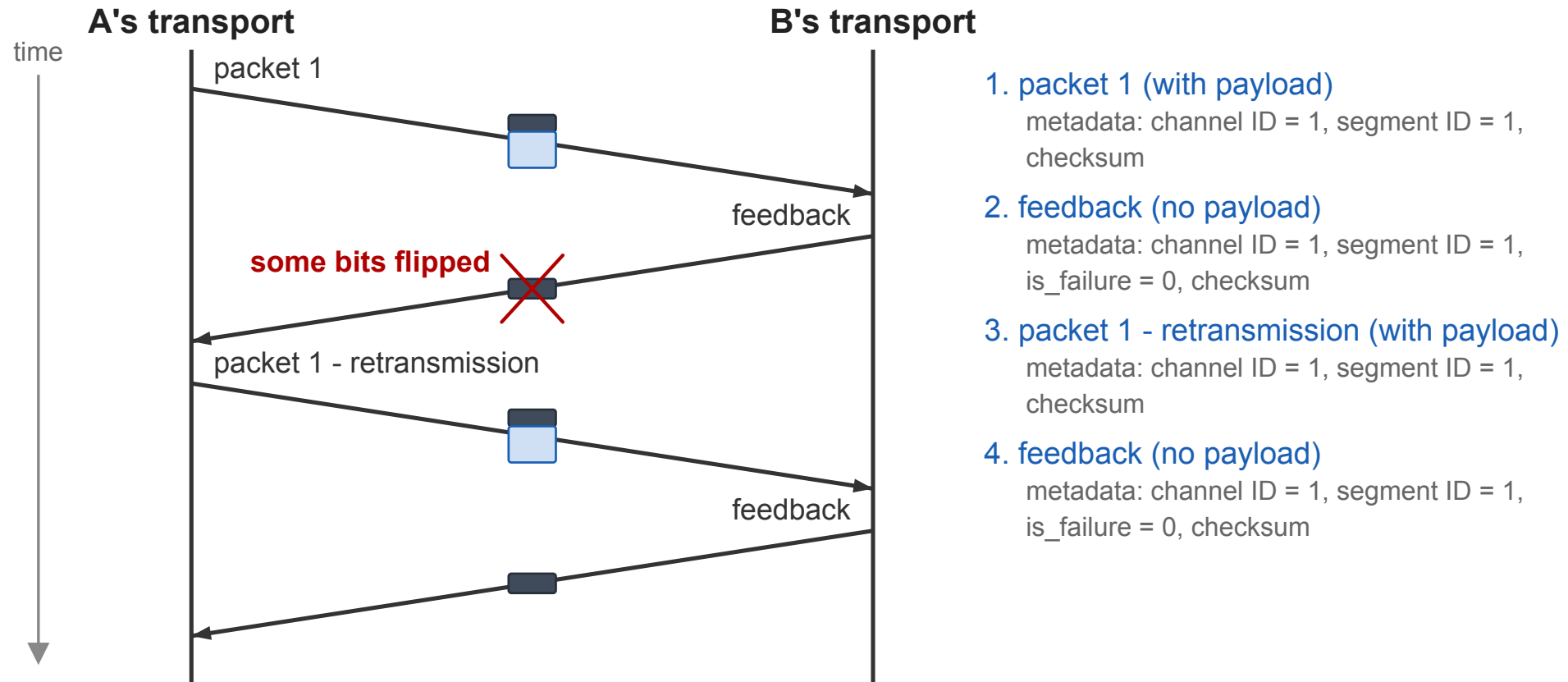
        while (true) {
            fb = from_lower();
            if (!corrupt(fb)) {
                if (fb.is_failure) {
                    to_lower(pkt);
                }
                else {
                    next_id[ch] = next_id[ch] + 1;
                    break;
                }
            }
            else {
                to_lower(pkt);
            }
        }
    }
}
```

## Transport at B (receiver)

```
void receive_loop() {
    while (true) {
        pkt = from_lower();
        if (corrupt(pkt)) {
            // channel ID, segment ID, is_failure
            fb = feedback(pkt.ch, pkt.seg_id, 1);
            to_lower(fb);
        }
        else {
            if (!has_been_received(pkt.seg_id)) {
                seg = strip_header(pkt);
                offset = pkt.seg_id * max_payload_size;
                recv_buf[pkt.ch].write(offset, seg);
            }
            fb = feedback(pkt.ch, pkt.seg_id, 0);
            to_lower(fb);
        }
    }
}
```



# Corrupted feedback - timeline (in-class solution)



# Can B receive duplicate segments?

## If yes, how does it deal with them? (in-class discussion)

---

Yes, discussed in class and captured in the in-class solution pseudocode

Reset

# Dealing with loss

---

Some packets will leave their source but never reach the destination.

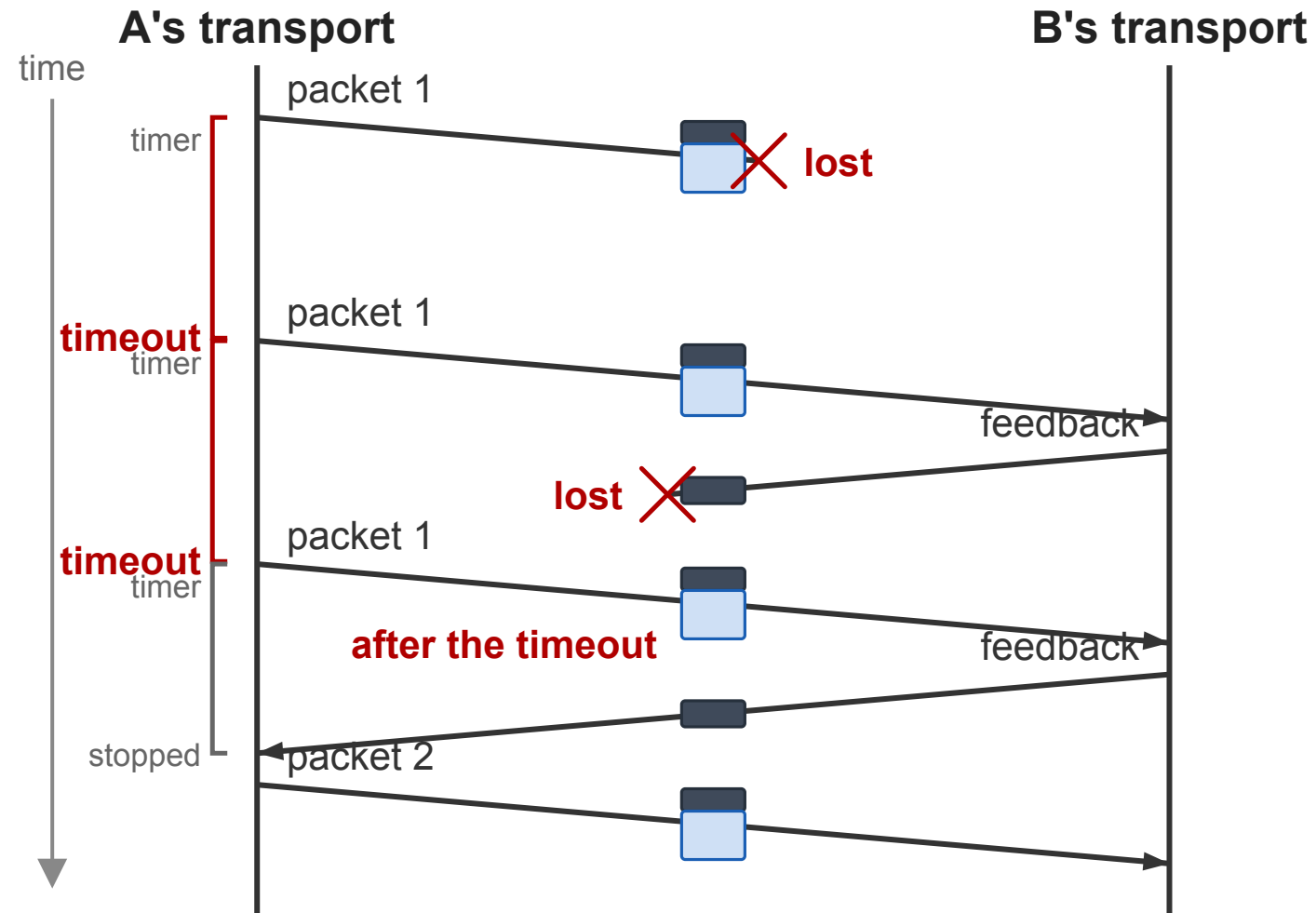
# Dealing with loss

---

Some packets will leave their source but never reach the destination.

How can an endpoint tell that a packet was lost?

# Lost data packet or lost feedback - timeline (in-class solution)



# Can A receive duplicate feedback?

## If yes, what do we do? (in-class discussion)

---

Yes, discussed in class.

Reset

# How do you know how long to wait? (in-class discussion)

---

Depends on the current round trip time (protocols continuously measure) and how many times the timer has expired back to back. Different protocols make different choices.

Reset

# What if I don't want 100% reliability? (in-class discussion)

---

(in class discussion):

- May not need feedback at all
- Send each packet several times without waiting for feedback (though this may defeat the performance benefits of not wanting 100% reliability)
- Ask for feedback only every few packets
- Do not handle every corner case
- Ignore lost packets, or limit the number of retries

Most of these suggestions pick a subset of the mechanisms, or do a mechanism only partly. One can imagine the application saying the minimum percentage of its data it needs reliably, and the transport adjusting how much it retries to meet that. Designing a transport protocol that allows trading off reliability in a general way that is beneficial for a wide range of applications is a hard problem and is not solved at the moment.

Reset



# Why should I care about how transport deals with an unreliable network?

---

- Won't the network stack take care of it for me?

-

# Why should I care about how transport deals with an unreliable network?

---

- Won't the network stack take care of it for me?
  - Yes, and no

# Why should I care about how transport deals with an unreliable network?

---

- Won't the network stack take care of it for me?
  - Yes, and no
- There are protocols designed with certain application requirements in mind. Making sense of how a transport protocol deals with an unreliable network helps you figure out which one is right for your application and your network.

# Why should I care about how transport deals with an unreliable network?

---

- Won't the network stack take care of it for me?
  - Yes, and no
- There are protocols designed with certain application requirements in mind. Making sense of how a transport protocol deals with an unreliable network helps you figure out which one is right for your application and your network.
- If you don't find a protocol that suits you, you may have to use a bare-bones transport that does nothing, and build similar mechanisms into your application to deal with the network's unreliability (e.g., a video call)

# What we have so far

---

- **One packet at a time:** the sender moves on to the next packet only once it is sure the current one has made it correctly to the receiver
- **Feedback for every packet:** the receiver sends feedback for every packet, positive if the packet was fine, negative if it detects the packet was corrupted using checksums
- **Timers:** the sender uses a timer to detect loss and retransmit. A timer may trigger and cause a retransmission even if the packet is not lost, e.g., if the packet is delayed longer than expected
- **Duplicates:** an unnecessary retransmission can deliver a segment more than once; the receiver uses the segment ID to deal with that

# What we have so far

---

- **One packet at a time:** the sender moves on to the next packet only once it is sure the current one has made it correctly to the receiver
- **Feedback for every packet:** the receiver sends feedback for every packet, positive if the packet was fine, negative if it detects the packet was corrupted using checksums
- **Timers:** the sender uses a timer to detect loss and retransmit. A timer may trigger and cause a retransmission even if the packet is not lost, e.g., if the packet is delayed longer than expected
- **Duplicates:** an unnecessary retransmission can deliver a segment more than once; the receiver uses the segment ID to deal with that

Quick check: what are three scenarios in which an unnecessary retransmission happens?

# Main mechanisms to deal with an unreliable network

---

- Receiver feedback
- Retransmission
- Timers
- Checksum

# Main mechanisms to deal with an unreliable network

---

- Receiver feedback
- Retransmission
- Timers
- Checksum

All transport protocols use a combination of these mechanisms and optimize for performance (next lecture)